ERP Inventory System

Database Design Document

This document describes the relational database schema for the ERP Inventory Management System. The application uses MySQL with JDBC and consists of two core tables: **users** and **inventory**.

Database Connection

Parameter	Value
Database Name	erp_system
Host	localhost
Port	3306
Driver	MySQL JDBC (com.mysql.cj.jdbc.Driver)
Default User	root
Connection Class	com.erp.db.DBConnection

Table 1: users

Stores login credentials and role assignments. Used by **UserDAO.login()** to authenticate users and by **AppSession** to hold the active session.

Column	Data Type	Constraint	Description
`id`	`INT`	PRIMARY KEY AUTO_INCREMENT	Unique row identifier
`username`	`VARCHAR(50)`	NOT NULL	Login credential
`password`	`VARCHAR(50)`	NOT NULL	Plain-text password (no hashing)
`role`	`VARCHAR(20)`	NOT NULL	'Admin' or 'Employee'

CREATE TABLE SQL

```
CREATE TABLE users (
    id       INT          PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50)  NOT NULL,
    password VARCHAR(50)  NOT NULL,
    role     VARCHAR(20)  NOT NULL
);

INSERT INTO users (username, password, role) VALUES
    ('admin',    'admin123', 'Admin'),
    ('employee', 'emp123',   'Employee');
```

Table 2: inventory

Stores product stock data. All CRUD operations are performed by **UserDAO** using JDBC PreparedStatements. Low-stock alerts are triggered when **current_stock <= reorder_level**.

Column	Data Type	Constraint	Description
`id`	`INT`	PRIMARY KEY AUTO_INCREMENT	Unique product identifier
`product_name`	`VARCHAR(100)`	NOT NULL	Product display name
`current_stock`	`INT`	NOT NULL	Units currently in stock
`reorder_level`	`INT`	NOT NULL	Alert fires when stock ≤ this
`price`	`DOUBLE`	NOT NULL	Unit price (INR ■)

CREATE TABLE SQL

```
CREATE TABLE inventory (
    id            INT          PRIMARY KEY AUTO_INCREMENT,
    product_name  VARCHAR(100) NOT NULL,
    current_stock INT          NOT NULL,
    reorder_level INT          NOT NULL,
    price         DOUBLE       NOT NULL
);
```

Table Relationship

The two tables are linked **logically** — there is no foreign key constraint in the schema. Role-based access is enforced entirely in application code (**DashboardController.initialize()**).

users	Cardinality	inventory	Enforced by
One Admin/Employee	`||—o{`	Many inventory rows	Application logic only (no FK constraint)

Query Reference (from UserDAO.java)

Operation	SQL Query
Login	`SELECT * FROM users WHERE username=? AND password=?`
Get all products	`SELECT * FROM inventory`
Add product	`INSERT INTO inventory(product_name, current_stock, reorder_level, price) VALUES(?,?,?,?)`
Update product	`UPDATE inventory SET product_name=?, current_stock=?, reorder_level=?, price=? WHERE id=?`
Delete product	`DELETE FROM inventory WHERE id=?`

Known Issues

Issue	Detail
DB name mismatch	README says CREATE DATABASE erpdb but DBConnection.java connects to erp_system. These
Plain-text passwords	Passwords stored as raw VARCHAR. No hashing (MD5 / BCrypt). A security risk in any real deploy

No foreign key	No FK from inventory to users — no audit trail of who added/changed a product.
No indexes	No secondary indexes defined. Large inventory tables will do full scans on every search.
Hardcoded credentials	root / Tiger hardcoded in DBConnection.java. Should use environment variables or a config file.

Recommended Schema Improvements

```
-- 1. Hash passwords (add a salt column)
ALTER TABLE users ADD COLUMN password_hash VARCHAR(255);
ALTER TABLE users ADD COLUMN salt         VARCHAR(64);

-- 2. Add audit columns to inventory
ALTER TABLE inventory ADD COLUMN created_by  INT REFERENCES users(id);
ALTER TABLE inventory ADD COLUMN updated_by  INT REFERENCES users(id);
ALTER TABLE inventory ADD COLUMN created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP;
ALTER TABLE inventory ADD COLUMN updated_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
                                             ON UPDATE CURRENT_TIMESTAMP;

-- 3. Add search index
CREATE INDEX idx_product_name ON inventory(product_name);

-- 4. Add stock constraint
ALTER TABLE inventory ADD CONSTRAINT chk_stock CHECK (current_stock >= 0);
```